

# Desarrollar un sistema en tiempo corto con metodologías ágiles, para el proyecto final de la materia (Examen Final)

Se debe reducir el tiempo de desarrollo sin sacrificar **la calidad** del sistema, el enfoque recomendado es combinar metodologías como **Scrum**, **Kanban** y prácticas de **DevOps**, apoyadas en herramientas modernas de desarrollo.

## 1. Definir claramente el problema

- Identificar la necesidad del cliente.
- Determinar los objetivos del sistema.
- Establecer el alcance mínimo (MVP - Producto Mínimo Viable).
- Identificar los usuarios finales.

**Resultado:** Documento breve de visión del proyecto.

## 2. Formar el equipo ágil

Asignar roles claros:

- Product Owner
- Scrum Master
- Desarrolladores
- Diseñador UX/UI
- Tester (QA)

En proyectos pequeños una persona puede **asumir más de un rol**.

## 3. Crear el Product Backlog

El Product Backlog contiene todas las funcionalidades que tendrá el sistema.

Ejemplo:

### Prioridad Historia de Usuario

|       |                     |
|-------|---------------------|
| Alta  | Registrar usuarios  |
| Alta  | Iniciar sesión      |
| Alta  | Registrar productos |
| Media | Generar reportes    |
| Baja  | Exportar a Excel    |

## 4. Escribir Historias de Usuario

Formato estándar:

**Como** administrador  
**Quiero** registrar productos  
**Para** controlar el inventario.

Cada historia debe tener criterios de aceptación.

## **5. Priorizar funcionalidades**

Aplicar la regla 80/20:

- Primero desarrollar lo indispensable.
- Después agregar funciones secundarias.

Esto permite entregar un sistema funcional rápidamente.

## **6. Planificar Sprints cortos**

Duración recomendada:

- 1 día.
- o máximo 2 días.

Cada Sprint debe producir una versión funcional del sistema.

Ejemplo:

Sprint 1

- Login
- Usuarios

Sprint 2

- Clientes
- Productos

Sprint 3

- Ventas

Sprint 4

- Reportes

## **7. Diseñar solo lo necesario**

Evitar documentación excesiva.

Realizar únicamente:

- Casos de uso
- Modelo de base de datos
- Mockups de pantallas
- Arquitectura general

## **8. Programar por módulos**

Dividir el sistema en componentes independientes.

Ejemplo:

- Seguridad
- Usuarios
- Inventario
- Ventas
- Reportes
- Configuración

Esto permite que varios desarrolladores trabajen simultáneamente.

## **9. Integración continua**

Cada cambio debe integrarse inmediatamente utilizando herramientas como:

- Git
- GitHub
- GitLab
- Azure DevOps

Beneficios:

- Menos errores
- Desarrollo paralelo
- Recuperación rápida

## **10. Pruebas durante todo el desarrollo**

No esperar al final.

Realizar:

- Pruebas unitarias
- Pruebas funcionales
- Pruebas de integración
- Pruebas con el cliente

## 11. Revisión del Sprint

Al finalizar cada Sprint:

- Mostrar el sistema funcionando.
- Recibir observaciones.
- Ajustar el Product Backlog.

## 12. Retrospectiva

El equipo responde:

- ¿Qué salió bien?
- ¿Qué salió mal?
- ¿Qué mejorar?

Estas mejoras se aplican en el siguiente Sprint.

## 13. Entrega incremental

En lugar de entregar todo al final, se libera una versión funcional después de cada Sprint.

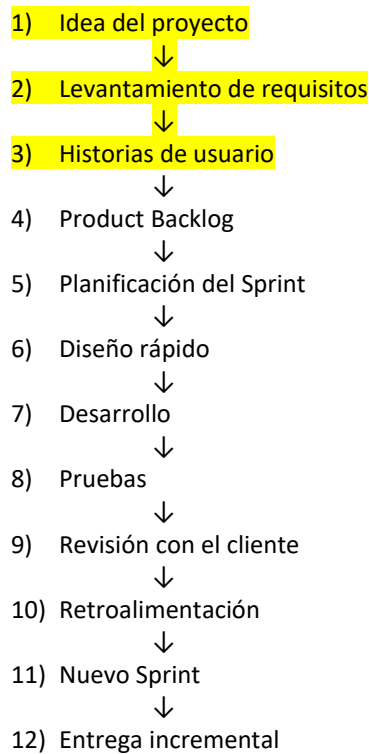
Ejemplo:

- Versión 1.0
- Versión 1.1
- Versión 1.2
- Versión final

## Herramientas recomendadas

| Actividad            | Herramientas                          |
|----------------------|---------------------------------------|
| Gestión del proyecto | Jira, Trello, Azure DevOps            |
| Diseño de interfaces | Figma, Balsamiq                       |
| Diagramación         | Draw.io, Lucidchart                   |
| Control de versiones | Git, GitHub, GitLab                   |
| Desarrollo           | Visual Studio Code, IntelliJ IDEA     |
| Base de datos        | PostgreSQL, MySQL, SQL Server         |
| Pruebas              | Postman, JUnit, Selenium              |
| Integración continua | GitHub Actions, Jenkins, GitLab CI/CD |

## Flujo ágil resumido



## Factores clave para desarrollar en poco tiempo

- Definir un alcance reducido (MVP).
- Mantener reuniones diarias de 15 minutos (Daily Scrum).
- Priorizar funcionalidades de mayor valor para el negocio.
- Automatizar pruebas e integración continua.
- Evitar documentación innecesaria.
- Involucrar al cliente durante todo el proceso.
- Reutilizar componentes, bibliotecas y plantillas.
- Trabajar con **equipos multidisciplinares y colaborativos**.

## Ejemplo de cronograma para un proyecto universitario (8 días)

| Días | Actividad                                                        |
|------|------------------------------------------------------------------|
| 1    | Definición del problema, requisitos y Product Backlog            |
| 2    | Diseño de arquitectura, base de datos y prototipos               |
| 3-4  | Sprint 1: Autenticación y gestión de usuarios                    |
| 5-6  | Sprint 2: Funcionalidades principales del negocio                |
| 7    | Sprint 3: Reportes, pruebas e integración                        |
| 8    | Pruebas finales, documentación mínima, despliegue y presentación |

Este enfoque permite obtener un sistema funcional en un plazo corto, con entregas frecuentes, retroalimentación continua y capacidad de adaptación a los cambios, **siguiendo las buenas prácticas de las metodologías ágiles modernas**.